

Correcciones del estimador de estado

TP4 – IMU MPU-6050 con micro-ROS · imu_state_estimator.py (rama dev) · Por qué se corrigió cada cosa y cómo funciona

Resumen

Al lanzar los tres métodos (Filtro Complementario, Madgwick y DMP), los frames en RViz se iban hacia lados distintos y a velocidades muy altas. Hay dos causas mezcladas:

- **Problemas del código**, que se pueden corregir: mensajes que se integraban dos veces y métodos que no arrancaban desde la misma orientación.
- **Un límite físico**, que no se corrige: estimar la posición integrando dos veces la aceleración de una IMU acumula error muy rápido.

#	Corrección	Qué resuelve
1	Descartar mensajes duplicados	Cada muestra se integraba dos veces: la deriva era 4 veces mayor.
2	Cero común para el DMP	El DMP arrancaba girado respecto de los otros dos métodos, y por eso no se podían comparar.
3	Tecla R = recalibrar	Permite volver a poner los tres métodos en el mismo cero en cualquier momento, y vuelve a estimar el bias del giróscopo.

Cómo se calcula la posición

Para entender las correcciones conviene recordar qué hace el nodo con cada mensaje de /imu/data_raw:

```
acc_mundo = R(q) · acc_medida - [0, 0, g]    # 1. pasar al mundo y restar la gravedad
vel      += acc_mundo · dt                    # 2. integrar → velocidad
pos      += vel · dt                          # 3. integrar otra vez → posición
```

q es la orientación (cuaternión) que estima cada método. Hay dos cosas que importan:

- El acelerómetro siempre mide la gravedad ($\approx 9,81 \text{ m/s}^2$). Para quedarse solo con el movimiento hay que restarla, y para eso hay que saber exactamente dónde está la vertical, o sea, conocer bien el roll y el pitch.
- Cualquier error en acc_mundo se integra dos veces. Un error constante a produce un error de posición de $\frac{1}{2} \cdot a \cdot t^2$: crece con el tiempo al cuadrado.

Corrección 1 - Descartar mensajes duplicados

¿Qué pasaba?

Al revisar el tópic se vio que /imu/data_raw tenía **dos publishers** del mismo nodo del ESP32 y llegaba a $\sim 200 \text{ Hz}$, cuando el firmware publica a $\sim 100 \text{ Hz}$. El agente micro-ROS había quedado con la sesión vieja del ESP32 después de un reinicio, así que cada mensaje llegaba dos veces, con el mismo timestamp.

El código calculaba el paso de tiempo así:

```
dt = t - self.t_prev
self.t_prev = t
if not 0.0 < dt < 0.1:    # timestamp inválido o salto
    dt = 0.01              # → se usa el período nominal
```

Para la copia duplicada, $dt = 0$. Ese caso caía en el `if` y se reemplazaba por 0,01 s. Resultado: **cada muestra se integraba dos veces**, como si hubiera pasado el doble de tiempo.

¿Por qué importa tanto?

Si el tiempo integrado es el doble del real, la velocidad acumula el doble de error y la posición, que crece con t^2 , **cuatro veces más**. Además, el eje de tiempo de los gráficos avanzaba al doble de la velocidad real.

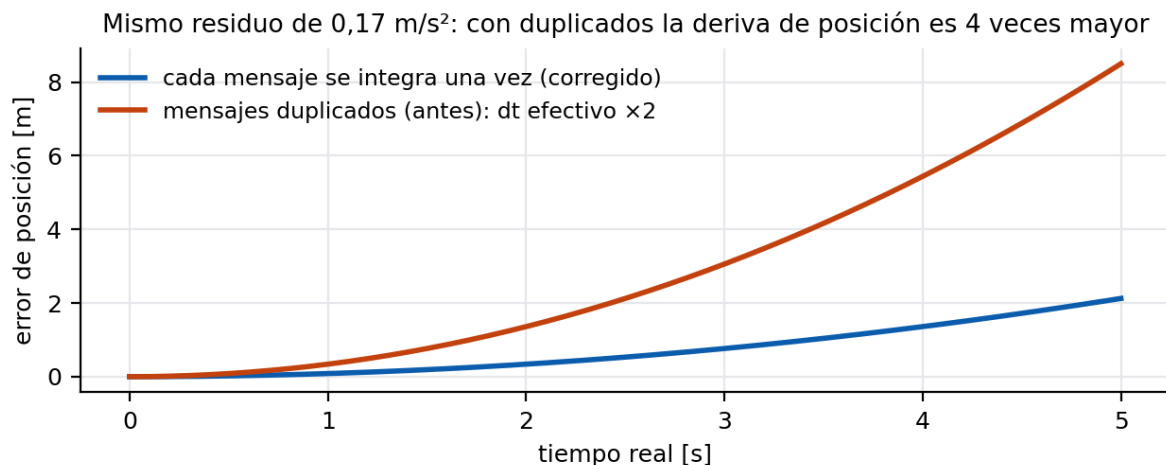


Figura 1. Error de posición para un residuo constante de 0,17 m/s² (equivalente a 1° de error de inclinación), con y sin duplicados.

La corrección

```
dt = t - self.t_prev
if dt <= 0.0:          # mensaje duplicado o desordenado: no integrarlo dos veces
    return
self.t_prev = t
if dt > 0.1:           # salto (se perdieron mensajes): usar período nominal
    dt = 0.01
```

Ahora un dt nulo o negativo significa "este mensaje ya lo procesé" y se ignora. El período nominal se usa solo para lo que realmente es un salto: cuando se pierden mensajes.

En el uso: el problema de fondo son las dos sesiones del ESP32 en el agente. Para evitarlas, lanzar primero `launch_estimation.sh` (que arranca el agente) y recién después apretar EN en el ESP32. Para verificar: `ros2 topic info /imu/data_raw` tiene que mostrar **1 publisher** y `ros2 topic hz` unos **100 Hz**.

Corrección 2 - Un cero común para los tres métodos

¿Qué pasaba?

Cada método definía su orientación inicial de manera distinta:

Método	Roll / pitch iniciales	Yaw inicial
Complementario	Del acelerómetro (promedio de la calibración)	0
Madgwick	Del acelerómetro (promedio de la calibración)	0
DMP (antes)	Los del chip	El que tenía el DMP al encenderse, con sus propios ejes

El nodo del DMP copiaba directamente el cuaternión del sensor, sin usar la calibración. Por eso en RViz `imu_link_dmp` aparecía girado respecto de `imu_link` e `imu_link_madg`. Además, su vertical no coincidía exactamente con la gravedad que mide el acelerómetro, así que la resta de la gravedad dejaba un residuo extra y su posición derivaba más rápido.

La idea

Al terminar la calibración, con el sensor quieto, se conocen dos orientaciones del mismo instante:

- `q_ref`: la del cero común (roll y pitch del acelerómetro, yaw = 0). Es la misma con la que arrancan el Complementario y Madgwick.
- `q_dmp0`: la que reporta el DMP en ese momento.

Se calcula una vez la rotación fija que lleva una a la otra, y después se aplica a cada cuaternión del DMP:

```
q_off = q_ref * q_dmp0-1      # se calcula una vez, al terminar la calibración
q(t)  = q_off * q_dmp(t)       # se aplica a cada mensaje
```

En el instante inicial, $q = q_{\text{off}} * q_{\text{dmp0}} = q_{\text{ref}}$, así que el DMP arranca exactamente en el cero común. Después, como `q_off` es constante, se conservan todos los giros que mide el DMP. Lo único que cambia es la referencia desde la que se miden. Para un cuaternión unitario, la inversa es el conjugado: $[w, -x, -y, -z]$.

¿Por qué roll y pitch del acelerómetro, y solo el yaw en 0?

- **Roll y pitch** tienen que ser los reales, porque de ellos depende restar bien la gravedad. Con el sensor quieto, el acelerómetro mide solo la gravedad y da la vertical de forma directa.
- **El yaw** no se puede observar con el MPU-6050, que no tiene magnetómetro. Ningún método conoce el "norte", así que se elige yaw = 0 como convención y queda igual para los tres.

La corrección en el código

```
# al terminar la calibración
self.q = q_from_accel(mean_acc)          # q_ref
if self.use_msg_q and self.q_dmp is not None:
    self.q_off = q_mult(self.q, q_conj(self.q_dmp)) # q_off = q_ref * q_dmp0-1

# en cada mensaje (rama DMP)
self.q = q_mult(self.q_off, self.q_dmp)
```

Verificación: se probó con mensajes simulados. Con el sensor inclinado 10° en roll y el DMP reportando un yaw arbitrario de 70° , después de la calibración el estimador del DMP da roll = 10° , pitch = 0° , yaw = 0° , igual que los otros dos métodos.

Corrección 3 - La tecla R recalibra

¿Qué pasaba?

La tecla R de los gráficos (y el servicio ~/reset) solo debía poner en cero la velocidad y la posición, pero usaba variables que no existían (self.vels, self.poss), así que fallaba. Además, aunque hubiera funcionado, no devolvía la orientación al cero común. Después de un rato, los tres métodos seguían con orientaciones distintas.

La corrección

Ahora R pide una **recalibración completa**. Durante unos 3 s, con el sensor quieto, el nodo vuelve a:

- estimar el bias del giróscopo, que cambia con la temperatura;
- medir |g| y la vertical;
- fijar el cero común, recalculando q_off en el caso del DMP;
- poner en cero la velocidad y la posición.

El pedido se hace con una bandera (self.recalib = True) y se aplica al comienzo del próximo mensaje dentro de on_imu. Esto es porque la tecla se procesa en el hilo de la ventana de matplotlib y los mensajes en el hilo de ROS. Si las dos partes modificaran el estado al mismo tiempo, se podría mezclar un estado a medio actualizar.

Uso: la tecla R recalibra solo el método de esa ventana. Para recalibrar los tres juntos, con el sensor quieto:

```
for n in complementary_estimator dmp_estimator madgwick_estimator; do ros2 service call  
/$n/reset std_srvs/srv/Empty; done
```

Lo que no se corrige: la deriva por doble integración

Aunque las tres correcciones estén aplicadas, **los tres métodos se van a seguir separando con el tiempo**. Esto es una limitación física de estimar la posición solo con una IMU, no un error del código.

Si la orientación estimada tiene un error de inclinación θ , la gravedad no se resta del todo y queda un residuo de $g \cdot \sin \theta$ en el plano horizontal. Integrado dos veces:

$$\text{error_posición}(t) = \frac{1}{2} \cdot g \cdot \sin \theta \cdot t^2$$

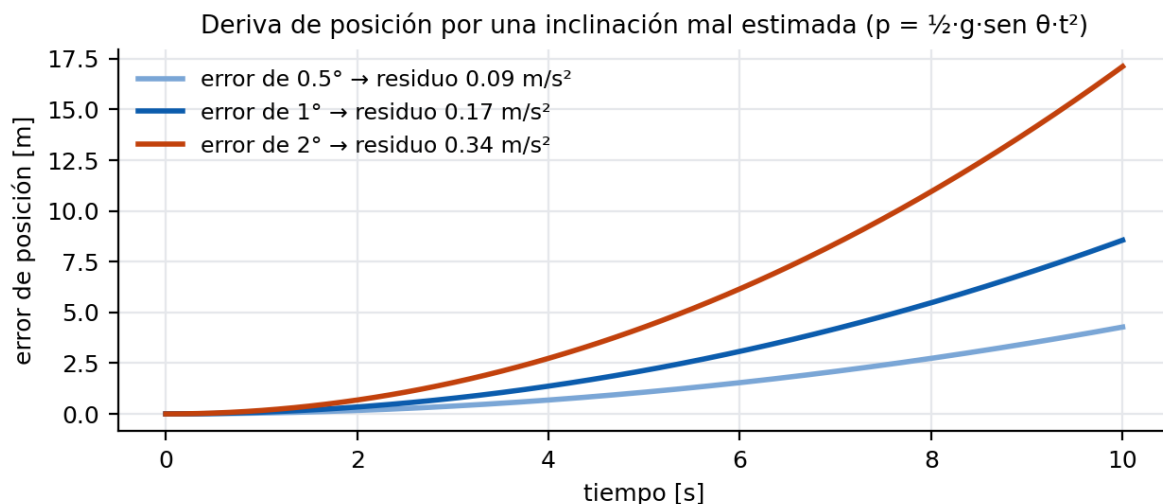


Figura 2. Con solo 1° de error de inclinación, la posición se equivoca unos 0,8 m a los 3 s y casi 9 m a los 10 s.

Cada método tiene un error de inclinación distinto, en una dirección distinta. Por eso cada frame "se cae" hacia otro lado. Que se separen es esperable. Lo interesante para el TP es **cuánto** se separa cada uno:

- **Complementario y Madgwick** corrigen roll y pitch con el acelerómetro, cada uno con su propia ganancia (kp, beta).
- **El DMP** hace su propia fusión dentro del chip, a mayor frecuencia.
- **El ZUPT** (velocidad = 0 cuando detecta reposo) frena la deriva de la velocidad mientras el sensor está quieto, pero no deshace la posición ya acumulada.

Sugerencia para el informe: recalibrar los tres a la vez, dejar el sensor 10 s quieto y anotar la posición final de cada método. Después repetir moviéndolo y devolviéndolo al punto de partida. Comparar con y sin ZUPT (`./launch_estimation.sh --no-zupt`).

Anexo - Errores que impedían que el nodo funcionara

Antes de estas correcciones, los gráficos y RViz quedaban vacíos aunque llegaban datos del ESP32. Se encontraron estos errores en la rama dev:

Error	Efecto	Corrección
on_imu llamaba a self.publish(), que no existe	Excepción en el primer mensaje después de la calibración: el nodo dejaba de procesar	Llamar a self.publish_tfs()
La posición de la odometría (Point) se asignaba con un Vector3	Excepción por tipo incorrecto al publicar	Asignar x, y, z por separado
spin_node tragaba las excepciones (except: pass)	Los errores anteriores no aparecían en ninguna parte	Mostrar el error con get_logger().error
RViz con Fixed Frame = map	"No tf data": los TF son odom → imu_link...	Fixed Frame = odom